

Hypothesise and Enhance

django-helpdesk

Team: Team 4
Members: Manoj. C, Meghana. S, Aseem

Date: Day 9 (Feature Enhancement Activity)
Repository: github.com/MANOFHATERS/DAY2-APPLIED-PROGRAMMING

Our task

We chose two enhancements from Appendix A and recorded our work in the same format as the worked example discussed in class.

We worked UI-first: ran the demo, signed in as each role, observed the screens, then went to the code. We did this because going straight to the code is not what the activity asks for.

Our selected enhancements:

- Enhancement 1 — A.2 Waiting for Customer: a new ticket status for tickets blocked on customer input.
- Enhancement 2 — A.1 Transfer a ticket to another queue: a dedicated Transfer action on the ticket detail page.

Our model evolves: M0 before, M1 after Enhancement 1, M2 after Enhancement 2. Changes for each step are described inside the worksheets.

Enhancement 1 Worksheet

Selected Enhancement 1: A.2 Waiting for Customer

0.1 Elaborate the customer need into a requirement

Customer need (Appendix A.2): agents cannot continue work on a ticket until the customer provides information. Today there is no clear ticket state for this. We would like agents to mark a ticket as Waiting for Customer and later resume work.

What we observed in the running UI:

- Ticket detail page, Respond tab: status radio buttons are Open, Resolved, Closed, Duplicate. No Waiting option.
- For a Resolved ticket: radio buttons change to Reopened, Resolved, Closed. Still no Waiting.
- Dashboard and ticket list use the same five labels.
- There is a Hold button, but it means an internal pause, not waiting for the customer.

An agent blocked on the customer has no honest way to show it. The ticket looks like active work, or has to be closed. This is the behaviour to change.

Changes we propose to X, U, and/or f:

X (ticket LTS state) adds a new status value:

X adds: Waiting for Customer

U is unchanged. The existing ticket-update action performs the new transitions. We add transitions to f:

f adds: (Open, Waiting) via update

f adds: (Reopened, Waiting) via update

f adds: (Waiting, Open) via update (resume)

f adds: (Waiting, Resolved | Closed | Duplicate) via update

Resolved and Closed cannot go directly to Waiting. A Closed ticket must be Reopened first, then can move to Waiting.

Important CBIF components/interactions affected:

- Ticket: new value for the status field. Component unchanged.
- FollowUp: each transition into or out of Waiting is recorded as a FollowUp, the same way every other status change is recorded today.
- Dashboard / Ticket list: read the ticket status. The new status sits in the same field, so existing queries pick it up automatically.

Behavioural transition table:

Current state	Action	Resulting state	Other effects / constraints
Open	update (set Waiting)	Waiting for Customer	FollowUp recorded; queue, owner, priority preserved
Reopened	update (set Waiting)	Waiting for Customer	FollowUp recorded; queue, owner, priority preserved
Waiting for Customer	update (set Open)	Open	Resume work. FollowUp recorded.
Waiting for Customer	update (set Resolved, Closed, or Duplicate)	Resolved, Closed, or Duplicate	Allowed if the customer reply makes one of these the right outcome
Resolved	update (set Waiting)	(rejected)	Not in flow. Use Reopen first.

Our detailed requirement:

An agent viewing an Open or Reopened ticket must be able to set its status to Waiting for Customer from the same Respond tab. The new status shows in the dashboard, ticket list, and ticket detail page. From Waiting, the agent can resume to Open or move to Resolved, Closed, or Duplicate. Queue, owner and priority are preserved. Resolved and Closed tickets cannot go directly to Waiting. Existing behaviour that must remain unchanged: the five original statuses, Hold / Unhold, the Edit Ticket form, and the way FollowUp entries are recorded.

0.2 Hypothesise about how the relevant existing behaviour works

Three explanations for how the status radio buttons are produced.

Hypothesis	Evidence for / against
H1 The radio buttons come from a single, fixed list of statuses shared by the whole application.	For: Every ticket screen we opened showed the same small set of labels. Suggests a single shared source.
H2 The radio buttons depend on the ticket's current status, via a per-status flow table that picks the valid destinations.	For: Ticket 1 (Open) showed Open, Resolved, Closed, Duplicate. Ticket 2 (Resolved) showed Reopened, Resolved, Closed. The two sets differ, so the current status influences the choices. Strongest explanation.
H3 The radio buttons come directly from the model's choices attribute, with no per-status filtering.	Against: If they were the full list, they would be the same on every ticket. They are not (see H2).

Our final explanation of how the relevant existing behaviour works:

The ticket detail page asks the ticket for the list of valid statuses from its current status, and renders that list as radio buttons. Adding a new status with the right flow entries is enough to make it appear in the UI. We did not have to change the template, dashboard, or ticket list.

0.3 Hypothesise about the change

Possible change	Advantages / problems
F1 Extend the shared status list (choices, open-status list, flow table, CSS class map) with the new Waiting status. No template or view changes.	Closest to the existing pattern. Adding the new status to the shared list automatically puts it in the right place on every screen. Risk: the rest of the codebase must read the shared list and not a hard-coded copy. We checked this against the running UI.
F2 Add a separate Mark as Waiting button on the ticket detail page with its own view and form.	Very visible in the UI. Problems: introduces a second, parallel path for changing status, conflicting with the existing update form. Also risks the dashboard and ticket list forgetting to count the new state.

Which approach will you implement, and why?

We chose F1. The existing system already has one shared place for the status list, per-status flow, and badge colours. Adding the new status there keeps the cause and effect visibly connected and does not introduce a second mechanism. F2 would have been more visible short term but would leave the codebase with two parallel mechanisms.

Change hypothesis:

Adding the Waiting status to the shared list and the right flow entries will make it appear as a radio button on Open and Reopened tickets, keep it off Resolved and Closed tickets, and let the dashboard and ticket list recognise the new state. The five existing statuses will not change.

0.4 Design tests before implementing

Test scenario	Expected result
T1 Open ticket 1, view the Respond tab.	Radio buttons include Open, Resolved, Closed, Duplicate, Waiting for Customer.
T2 Set ticket 1 to Waiting for Customer via the update form.	Status becomes Waiting for Customer. Ticket detail, list and dashboard show the new status. A FollowUp is recorded.
T3 From the Waiting ticket, set status back to Open.	Status becomes Open. Transition recorded as a FollowUp. Queue, owner and priority unchanged from before T2.
T4 View ticket 2 (Resolved) and inspect the radio buttons.	Radio buttons are Reopened, Resolved, Closed. Waiting does NOT appear (flow restriction).
T5 Edit ticket 1's title only, leaving status on Open.	Status remains Open. The new rule is not triggered by unrelated updates. Regression check.

0.5 Implement, validate, and refine the model

Final result and anything we had to revise:

All five scenarios passed on the first attempt after adding the new status to the four shared places (choices, open-status list, flow table, CSS class map). We did not have to revise the change. T1 showed the new radio button. T2 moved the ticket to Waiting and recorded the FollowUp. T3 returned the ticket to Open with queue, owner and priority preserved. T4 confirmed a Resolved ticket does not show the Waiting option. T5 confirmed an unrelated edit leaves status alone.

We also ran the project's existing test suite. All previously passing tests still pass.

Important code locations investigated and what we learned:

- Shared helpdesk status settings module: where the existing status constants, choices, open-status list, flow table, and CSS class map are defined. Confirmed H2.
- Ticket detail template: renders the radio buttons by iterating over the per-status flow list. Adding the new status to the flow table is enough.
- Ticket model: status field and get_allowed_status_flow property. Confirmed the per-status flow is the source the UI uses.

How M0 changed to become M1:

M0 modelled ticket status as the set {Open, Reopened, Resolved, Closed, Duplicate} with a per-status flow table. M1 adds the new status Waiting for Customer to that set, adds the transitions Open to Waiting and Reopened to Waiting, and adds the resumption transitions Waiting to Open / Resolved / Closed / Duplicate. The LTS action set U is unchanged. The CBIF component boundaries are also unchanged; only the Ticket component's status value set and the status flow inside it changed.

Git tag for this point: enhancement-1-complete

Enhancement 2 Worksheet

Enhancement 2 starts from the system after Enhancement 1. We use M1 as the starting model and the code tagged enhancement-1-complete as the starting implementation.

Selected Enhancement 2: A.1 Transfer a ticket to another queue

0.6 Elaborate the customer need into a requirement

Customer need (Appendix A.1): support teams sometimes need to hand a ticket to another queue. Today, changing the queue is part of editing a ticket. We would like Transfer to be a clear ticket action of its own. A transfer should move the ticket to the selected destination queue, work sensibly with the destination queue's default owner and the ticket's existing owner, and be visible in the ticket's history.

What we observed in the running UI:

- Actions dropdown on the ticket detail page contains only Edit, Hold, Delete. No Transfer.
- The only way to move a ticket to a different queue is via Edit, which opens a large form with Title, Queue, Submitter E-Mail, Description, Priority, Due on, Secret key, KB item, Merged to. The Queue field is the third field.
- Submitting the Edit form after changing the queue produces a generic queue-change follow-up. It does not look like a deliberate transfer.
- The ticket detail page shows the Queue field read-only in the description card. The agent has to leave the page to change the queue.

Changes we propose to X, U, and/or f:

X is unchanged. Queue is already part of the ticket state in M1. U gains a new action, and f gains the transitions performed by that action.

U adds: Transfer(ticket_id, destination_queue, comment)

f adds: (queue=q, ...) --Transfer(-->q') (queue=q', ...)

Transfer takes a destination queue and an optional comment. The transition leaves all other ticket state (status, priority, owner, due date, etc.) unchanged. The destination queue is restricted to the queues the agent can access, minus the current queue, so the action cannot be a no-op.

Default owner: if the destination queue has a default owner and the ticket is currently unassigned, the ticket is assigned to that default owner as part of the transfer. If the ticket is already assigned, we leave the owner alone.

Important CBIF components/interactions affected:

- Ticket: queue field is changed; owner field may be changed. No new field.
- Queue: read-only role here. The destination queue provides its default owner if needed. The Queue component itself is unchanged.
- FollowUp: a new follow-up entry is created with the title "Ticket transferred from <src> to <dst>", the agent as user, the current time, the optional comment, and a public flag. A TicketChange entry records the old and new queue values. This is the same pattern the existing Hold / Unhold and status-change actions use.
- Interaction with Enhancement 1: a Waiting-for-Customer ticket can be transferred too. The transfer leaves the ticket in the Waiting status. We deliberately did not change the status during transfer.

Behavioural transition table:

Current state	Action	Resulting state	Other effects / constraints
Ticket with queue=q	Transfer(destination=q', comment)	Ticket with queue=q'	FollowUp "transferred from q to q'" recorded; status and priority unchanged; owner set to default owner of q' if ticket was unassigned
Ticket with queue=q	Transfer(destination=q)	(rejected)	Same-queue transfer is not allowed; the form is shown again with an error
Ticket with queue=q	Transfer(destination=q', comment)	Ticket with queue=q'	If q' has no default owner or ticket already has an owner, owner is preserved

Our detailed requirement:

On the ticket detail page, the Actions dropdown must contain a new Transfer entry that opens a dedicated transfer page. The page shows the ticket's current queue, a dropdown of valid destination queues, an optional comment field, and a Transfer ticket button. On submit, the ticket's queue is updated, a FollowUp titled "Ticket transferred from <src> to <dst>" is created with the agent and the optional comment, and a TicketChange row records the old and new queue. If the destination queue has a default owner and the ticket was previously unassigned, the ticket is assigned to that default owner; otherwise the existing owner is preserved. Status and priority are never changed by Transfer. Submitting the current queue is rejected with an error. The existing Edit Ticket form must still allow changing the queue alongside other fields; all Enhancement 1 behaviour must continue to work.

0.7 Hypothesise about how the relevant existing behaviour works

Hypothesis	Evidence for / against
H1 The queue can only be changed via the Edit Ticket form, which saves the entire ticket and produces a single generic follow-up for the whole edit.	For: In the running UI the only way to change the queue was via Edit. The only history entry was a generic edit follow-up. No dedicated Transfer button anywhere. Strongest explanation.
H2 The queue can also be changed through the small ticket-update form on the ticket detail page (the one used for status and owner changes).	Against: The small update form does have a hidden queue field, but it is hidden precisely so that queue changes go through the Edit form. The UI does not expose queue as an editable field there.

Hypothesis	Evidence for / against
H3 There is already a Transfer action somewhere that we missed, and the customer need is asking us to expose it.	Against: We opened every link in the navigation sidebar and every entry in the Actions dropdown. None of them was a Transfer action. So if it exists, it is not reachable from the UI.

Our final explanation of how the relevant existing behaviour works:

Today the queue is changed only as part of editing a ticket. The Edit Ticket form treats the queue as one of several editable fields and saves the whole ticket; the history records the change as a generic queue change inside an edit. There is no dedicated Transfer action in the UI and no labelled follow-up for transfers. This is exactly what the customer need describes as the behaviour to change.

0.8 Hypothesise about the change

Possible change	Advantages / problems
F1 Add a dedicated Transfer view and URL, a small Transfer template with just a destination-queue dropdown and an optional comment, and a Transfer link in the Actions dropdown. The view writes a labelled FollowUp and a TicketChange row.	Closest to the existing pattern (Hold / Unhold is implemented the same way). The action is visible in the UI and in history. Risk: a new view is a new code path; we must make sure it does not bypass the queue-change rules the Edit form enforces.
F2 Add a Transfer button that simply navigates to the Edit form with the queue field highlighted, and relabel queue-only edits as transfers.	Reuses the existing form. Problems: the action is still inside the Edit form, which is what the customer need says is wrong. The heuristic is fragile. Rejected.

Which approach will you implement, and why?

We chose F1. The customer need explicitly says Transfer should be a clear action of its own, not part of editing a ticket. F1 makes Transfer its own action in the Actions menu, its own page, its own view, and its own labelled follow-up in history. F1 also matches the way the existing Hold action is implemented.

Change hypothesis:

Adding a dedicated Transfer view, a small Transfer template, a Transfer link in the Actions dropdown, and a labelled follow-up will make Transfer visible as its own action, move the ticket to the selected queue in one step, record the transfer as a labelled FollowUp in history, leave status and priority unchanged, and not affect the existing Edit Ticket form.

0.9 Design tests before implementing

These tests cover Enhancement 2 and also include regression tests for Enhancement 1.

Test scenario	Expected result
T1 Open ticket 1, open the Actions dropdown.	Dropdown contains Edit, Transfer, Hold, Delete. Transfer is its own action.
T2 Click Transfer, choose People Operations, write a comment, submit.	Ticket 1 queue becomes People Operations. Status remains Open. Priority remains Normal. A FollowUp titled "Ticket transferred from IT Support to People Operations" appears in history.
T3 From People Operations, click Transfer again, choose IT Support, submit.	Ticket 1 queue becomes IT Support. A second labelled transfer FollowUp appears. TicketChange rows record the queue moving back.
T4 Click Transfer on ticket 1 (in IT Support) and submit without choosing a destination, or choose IT Support itself.	Form is shown again with an error. Ticket 1 queue remains IT Support. Same-queue transfer rejected.
T5 Mark ticket 1 as Waiting for Customer (Enhancement 1), then Transfer it to People Operations.	Ticket is in People Operations queue and still has Waiting for Customer status. Both enhancements coexist.

0.10 Implement, validate, and refine the model

Final result and anything we had to revise:

All five scenarios passed. T1 showed the new Transfer entry in the Actions dropdown. T2 transferred ticket 1 from IT Support to People Operations and recorded the labelled follow-up with the comment, leaving status and priority unchanged. T3 transferred back and recorded a second labelled follow-up. T4 rejected the same-queue transfer and re-displayed the form with an error. T5 confirmed a Waiting-for-Customer ticket can be transferred without losing its Waiting status.

One revision: our first Transfer view did not explicitly exclude the current queue on the POST path, only on the dropdown shown on GET. We noticed this while writing T4 and added the same check on POST so a crafted request cannot transfer a ticket to its own queue. After the revision, T4 passed.

We also ran the project's existing test suite and added new tests for both enhancements. The existing tests still pass and the new tests pass.

Important code locations investigated and what we learned:

- Ticket detail template and the included description card template: where the Actions dropdown is rendered. Confirmed H1 - only Edit, Hold / Unhold and Delete.
- Existing Hold / Unhold view in the staff views module: the closest pattern to what we needed. A small dedicated view, a labelled FollowUp, a TicketChange row, then redirect. We used the same pattern for Transfer.
- Existing Edit Ticket view and form: confirmed the queue is one of many fields on the general edit form, which is what the customer need asks us to stop relying on. We did not change the Edit form.
- URL configuration: where we added the new transfer URL pattern, next to the existing edit and hold patterns.

How M1 changed to become M2:

M1 already had Ticket with a queue field and FollowUp with a labelled title and an optional comment. M2 adds the Transfer action to U with the transition described in 0.6. No new state component is added to X. The CBIF component boundaries are unchanged; only the Ticket component gains a new way to have its queue field changed, and the FollowUp component gains a new labelled title it can carry. The interaction with Enhancement 1 is captured in M2 by noting that Transfer does not change status, so a Waiting ticket stays Waiting across a transfer.

Git tag for this point: enhancement-2-complete

Final Reflection

1. What did you learn while doing Enhancement 1 that made Enhancement 2 easier, or changed how you approached it?

Enhancement 1 taught us that the existing system has a single shared place that defines the ticket status list, the per-status flow, and the badge colours, and that the UI reads from that shared place. That made us start Enhancement 2 by looking for the same kind of shared place for ticket actions. We found that the Actions dropdown on the ticket detail page was the equivalent shared place, and the existing Hold action was the closest template. So we wrote Enhancement 2 to follow the same pattern rather than reaching for a more elaborate design.

2. Did Enhancement 2 force you to revise anything you thought you understood after Enhancement 1? Explain.

Yes. After Enhancement 1 we had a working assumption that adding a new behaviour was always a matter of extending a shared list and the UI would pick it up. Enhancement 2 forced us to revise that: a new action on the ticket detail page has to be added explicitly to the Actions dropdown, routed through a new URL, and rendered by a new view. We also had to revise our assumption that the same-queue check was only needed on the dropdown shown to the user - we added it to the POST handler after T4 made the gap visible.

3. Identify one part of M2 for which you now have stronger evidence than you had in M0. What is that evidence?

The CBIF component boundary between Ticket and FollowUp. In M0 we modelled FollowUp as a passive log of past events. After both enhancements we have stronger evidence that FollowUp is the shared mechanism the codebase uses to make any state change visible in history - both the new status transitions and the new Transfer action write a labelled FollowUp and a TicketChange row, exactly the way the existing Hold / Unhold action does. The evidence is that we did not need to introduce any new history mechanism for either enhancement; we just used the existing FollowUp + TicketChange pattern with a new labelled title.

AI Use and Declaration

We used AI as a tool while completing this activity. The decisions about how to interpret the customer needs, which hypotheses to keep, which implementation approach to choose, which test scenarios to write, and how to revise the change after a test failed were made by our team. Important claims in this worksheet were checked against the running application, the project's existing test suite, and our own new tests.

We used AI to:

- locate where the ticket status list and per-status flow are defined, after we had already observed the per-status behaviour in the UI.
- explain the existing Hold / Unhold view and template, which we used as the pattern for the new Transfer view.
- suggest edge cases for our tests, including the same-queue transfer case (T4 for Enhancement 2).
- suggest two possible places where the new Waiting status could be implemented (our F1 and F2).

AI influenced our requirement decisions or implementation by:

suggesting that putting the new Waiting rule in the shared status list would propagate it everywhere through the UI. We used that suggestion, but only after checking against the running UI that the radio buttons really are produced from the shared list. AI also suggested F2 (a separate button with its own view); we did not use that suggestion because, after walking through the UI and discussing the model, we concluded it would create a second parallel path for status changes.

We checked these suggestions or claims by:

- walking through the running application as customer, agent and manager before writing anything down, and writing the UI observations into this worksheet as the evidence in the hypothesis tables.
- running the project's existing test suite after each enhancement and confirming that previously passing tests still pass.
- writing our own new tests for both enhancements and running them. The new tests live in `tests/test_team4_enhancements.py` in the repository.
- capturing screenshots of each test scenario running in the live UI; these are used as evidence in the demo recording that accompanies this worksheet.

The important decisions our team made ourselves were:

- the choice of the two enhancements (A.2 Waiting for Customer and A.1 Transfer).
- the flow restriction that Resolved and Closed tickets cannot go directly to Waiting.
- the rule that the destination queue's default owner is used only when the ticket is currently unassigned.
- the decision to reject same-queue transfers on the POST path as well as the GET path, which we added after our own test T4 showed the gap.
- the final detailed requirements, the test scenarios, the implementation choices (F1 in both cases), and the explanations in the reflection.